# ТЕМА 6. ДОПОЛНИТЕЛЬНАЯ ПРАКТИКА

## ЛЕКЦИЯ 6.25. Методологии CSS.

*Общая информация. БЭМ. SMACSS*

### Общая информация

С увеличением количества наших знаний, создаваемые проекты становятся сложнее, количество кода увеличивается. А значит, нам нужно учитывать не только сиюминутный результат, но и перспективы дальнейшего с ним взаимодействия.

Итак, если код большой, со временем в него вносятся изменения, а организация отсутствует, то это приводит к тому, что разработчик или даже целая команда начинают добавлять новые стили в конец документа.

Разработчики перестают удалять куски кода и изменять уже имеющиеся. Основная причина в том, что зачастую удаление и редактирование стилей может привести к неожиданным последствиям, а также к поломке дизайна в отдельных местах. Но это проигрышная стратегия, которая может привести к дублированию кода, проблемам специфичности, а переопределение правил станет настоящей русской рулеткой.

Именно поэтому, уже на старте работы над проектом мы должны определить набор правил, согласно которым будем обращаться к элементам и выстраивать структуру стилей страниц.

Такие наборы правил называют методологиями CSS. Следование методологии CSS гарантирует, что все, кто участвует в процессе разработки, говорят на одном языке. Это особенно важно в больших проектах с несколькими людьми,


©Pechora-PRO

поскольку упрощает масштабирование программного обеспечения и позволяет новым членам команды быстро освоить кодовую базу.

Есть несколько популярных методологий, у каждой свои особенности использования, достоинства и недостатки.

## БЭМ. Блок. Элемент. Модификатор

БЭМ (Блок, Элемент, Модификатор) — компонентный подход к веб-разработке. В его основе лежит принцип разделения интерфейса на независимые блоки. Он позволяет легко и быстро разрабатывать интерфейсы любой сложности и повторно использовать существующий код.

### Блок

Блок - функционально независимый компонент страницы, который может быть повторно использован. Например, форма поиска или карточка товара.

В HTML блоки выделяют с помощью атрибута **class**.

1. Название блока должно описывать его суть («что это?» — «меню»: menu, «кнопка»: button), а не состояние («какой, как выглядит?» — «красный»: red, «большой»: big).

Пример

```
<!-- Верно. Семантически осмысленный блок `error` -->
<div class="error"></div>
<!-- Неверно. Описывается внешний вид -->
<div class="red-text"></div>


<!-- Верно. Семантически осмысленный блок `card` -->
<div class="card"></div>
<!-- Неверно. Описывается внешний вид -->
<div class="big-rectangle"></div>

<!-- Верно. Семантически осмысленный блок `avatar` -->
```


©Pechora-PRO

```
<div class="avatar"></div>
<!-- Неверно. Описывается внешний вид -->
<div class="round-photo"></div>
```

2. При проектировании структуры важно, чтобы блок не влиял на свое окружение, т. е. блоку не следует задавать внешнюю геометрию (в виде отступов, границ, влияющих на размеры) и позиционирование. Значит необходимо четко разделить структурные и семантические элементы страницы.
3. Также в CSS по БЭМ не рекомендуется использовать селекторы тегов или id.

*Таким образом, обеспечивается независимость, при которой возможно повторное использование или перенос блоков с места на место без потери его оформления. При этом блоки можно вкладывать друг в друга на любой уровень.*

**Элемент**

Элемент – это составная часть блока, которая не может использоваться в отрыве от него.

1. Название элемента характеризует смысл («что это?» — «пункт»: item, «текст»: text), а не состояние («какой, как выглядит?» — «красный»: red, «большой»: big).
2. Имя элемента отделяется от имени блока двумя подчеркиваниями (__). Структура полного имени элемента соответствует схеме:

**имя-блока__имя-элемента.**

```
<!-- Блок `search-form` -->
<form class="search-form">
    <!-- Элемент `input` блока `search-form` -->
    <input class="search-form__input">
    <!-- Элемент `button` блока `search-form` -->
    <button class="search-form__button">Найти</button>
```


©Pechora-PRO

```
</form>

<!-- Блок `card` -->
<div class="card">
    <!-- Элемент `photo` блока `card` -->
    <img src="..." alt="..." class="card__photo">
    <!-- Элемент `text` блока `card` -->
    <span class="card__text"></span>
</div>
```

3. Элементы можно вкладывать друг в друга на любой уровень. При этом элемент — всегда часть блока, а не другого элемента. Это означает, что в названии элементов **нельзя** прописывать иерархию вида **block__elem1__elem2**.

```
<form class="search-form">
  <div class="search-form__content">
    <!-- Неверно! -->
    <input class="search-form__content__input">
    <!-- Замените на:
           `search-form__input` или `search-form__content-input` -->

    <!-- Неверно! -->
    <button class="search-form__content__button">
        Найти
    </button>
    <!-- Замените на:
           `search-form__button` или `search-form__content-button` -->
  </div>
</form>
```

Блок может иметь вложенную структуру элементов в DOM-дереве:

```
<div class="block">
    <div class="block__elem1">
        <div class="block__elem2">
            <div class="block__elem3"></div>
        </div>
    </div>
</div>
```


©Pechora-PRO

Однако эта же структура блока в методологии БЭМ всегда будет представлена плоским списком элементов:

```
.block {}
.block__elem1 {}
.block__elem2 {}
.block__elem3 {}
```

Это позволяет изменять DOM-структуру блока без внесения правок в коде каждого отдельного элемента. Структура блока меняется, а правила для элементов и их названия остаются прежними.

4. Элемент — всегда часть блока и не должен использоваться отдельно от него. При этом элемент — необязательный компонент блока. Не у всех блоков должны быть элементы.

**Блок или элемент?**

Первое время при использовании методологии сложно определиться что создавать: блок или элемент.

Если фрагмент кода может использоваться повторно и не зависит от реализации других компонентов страницы – **создаем блок.**

Если фрагмент кода не может использоваться самостоятельно, без родительской сущности (блока) – **создаем элемент.**

Исключение составляют элементы, реализация которых для упрощения разработки требует разделения на более мелкие части — подэлементы. В БЭМ-методологии нельзя создавать элементы элементов. В подобном случае вместо элемента необходимо создавать служебный блок.

**Модификатор**


©Pechora-PRO

Модификатор – это сущность, определяющая внешний вид, состояние или поведение блока либо элемента.

1. Название модификатора характеризует **внешний вид** («какой размер?», «какая тема?» и т. п. — «размер»: size_s, «тема»: theme_islands), **состояние** («чем отличается от прочих?» — «отключен»: disabled, «фокусированный»: focused) и **поведение** («как ведет себя?», «как взаимодействует с пользователем?» — «направление»: directions_left-top).
2. Имя модификатора отделяется от имени блока или элемента одним подчеркиванием (_). Структура полного имени модификатора соответствует схеме:

**имя-блока_имя-модификатора;**
или
**имя-блока__имя-элемента_имя-модификатора.**

Есть два типа модификаторов: булевый и ключ-значение.

**Булевый** используют, когда важно только наличие или отсутствие модификатора, а его значение несущественно. Например, «активен»: active. Считается, что при наличии булевого модификатора у сущности его значение равно true.

**Ключ-значение** используют, когда важно значение модификатора. Например, «меню с темой оформления islands»: menu_theme_islands.

Тогда структура полного имени модификатора будет соответствовать схеме:

**имя-блока_имя-модификатора_значение-модификатора;**
или
**имя-блока__имя-элемента_имя-модификатора_значение-модификатора.**


©Pechora-PRO

Невозможно одновременно использовать два одинаковых модификатора с разными значениями

**Важно!** Модификатор нельзя использовать самостоятельно

С точки зрения БЭМ-методологии модификатор не может использоваться в отрыве от модифицируемого блока или элемента. Модификатор должен изменять вид, поведение или состояние сущности, а не заменять ее.

**Микс**

Прием, позволяющий использовать разные БЭМ-сущности на одном DOM-узле.

Миксы позволяют:

- совмещать поведение и стили нескольких сущностей без дублирования кода;
- создавать семантически новые компоненты интерфейса на основе имеющихся.

```
<!-- Блок `header` -->
<div class="header">
    <!-- К блоку `search-form` примиксован элемент `search-form` блока `header`-->
    <div class="search-form header__search-form"></div>
</div>
```

В данном примере мы совместили поведение и стили блока **search-form** и элемента **search-form** блока **header**. Такой подход позволяет нам задать внешнюю геометрию и позиционирование в элементе **header__search-form**, а сам блок **search-form** оставить универсальным. Таким образом, блок можно использовать в любом другом окружении, потому что он не специфицирует никакие отступы. Это позволяет нам говорить о его независимости.

**Файловая структура**


©Pechora-PRO

Принятый в методологии БЭМ компонентный подход применяется и к организации проектов в файловой структуре. Реализации блоков, элементов и модификаторов делятся на независимые файлы-технологии, что позволяет нам подключать их опционально.

Особенности:

1. Один блок — одна директория.
2. Имена блока и его директории совпадают.
    - блок header — директория header/
    - блок menu — директория menu/
3. Реализация блока разделяется на отдельные файлы-технологии.
    - header.css
    - header.js.
4. Директория блока является корневой для поддиректорий соответствующих ему элементов и модификаторов.
5. Имена директорий элементов начинаются с двойного подчеркивания (__).
    - header/__logo/
    - menu/__item/
6. Имена директорий модификаторов начинаются с одинарного подчеркивания (_).
    - header/_fixed/
    - menu/_theme_islands/
7. Реализации элементов и модификаторов разделяются на отдельные файлы-технологии.
    - header__input.js
    - header_theme_islands.css.

Такая файловая структура позволяет легко поддерживать и повторно использовать код. А также предполагает, что в готовый код будет собираться в общие файлы проекта.


©Pechora-PRO

| | |
|---|---|
| search-form/ | Директория блока search-form |
| __input/ | Поддиректория элемента search-form__input |
| search-form__input.css<br>search-form__input.js | Реализация элемента search-form__input в технологии CSS<br>Реализация элемента search-form__input в технологии JavaScript |
| __button/ | Поддиректория элемента search-form__button |
| search-form__button.css<br>search-form__button.js | |
| _theme/ | Поддиректория модификатора search-form_theme |
| search-form_theme_islands.css | Реализация блока search-form, имеющего модификатор theme со значением islands в технологии CSS |
| search-form_theme_lite.css | Реализация блока search-form, имеющего модификатор theme со значением lite в технологии CSS |
| search-form.css<br>search-form.js | Реализация блока search-form в технологии CSS<br>Реализация блока search-form в технологии JavaScript |

Придерживаться рекомендуемой файловой структуры не обязательно. Вы можете использовать любую альтернативную структуру проекта, соответствующую принципам организации файловой структуры БЭМ.

**Важно! Важно!** В лекции представили краткую справку о методологии. Подробная информация на официальном портале https://ru.bem.info/

## SMACSS

SMACSS — это набор правил для категоризации набора правил CSS. Следуя методологии SMACSS, мы можем разделить наборы правил CSS на пять категорий:

- Base / Базовые
- Layout / Макет
- Module / Модуль


©Pechora-PRO

- State / Состояние
- Theme / Тема

В отличие от БЭМ, методология определяет способ написания стилей, не ставя при этом жесткие рамки. Автор предлагает самостоятельно определить для себя необходимую степень погружения в методологию.

**Base / Базовые**

Базовое правило применяется к элементу с помощью селектора элемента или селектора-потомка, а также любых псевдоклассов. Оно не включает в себя селекторы классов или идентификаторов. Базовое правило определяет стиль по умолчанию, определяющий, как этот элемент должен выглядеть во всех случаях на странице.

Базовые стили включают настройку размеров заголовков, стилей ссылок по умолчанию, стилей шрифтов по умолчанию и фона основного текста.

```
body, form {
    margin: 0;
    padding: 0;
}

a { color: #039; }

a:hover { color: #03F; }
```

Создатель методологии также рекомендует обязательно определять фон телу документа. Так мы обезопасим себя от нестандартных пользовательских настроек.

К базовым также относят стили сброса настроек CSS Reset.

**Layout / Макет**


©Pechora-PRO

Стили макета можно разделить на основные и второстепенные на основе их повторного использования.

Основные стили макета, такие как header, footer, боковое меню, body и т.д., традиционно оформляются с использованием селекторов идентификаторов. Чтобы избежать проблем со специфичностью, можно заменить из на классы с соответствующими названиями.

```
#header,
#article,
#footer {
    width: 960px;
    margin: auto;
}

#article {
    border: solid #CCC;
    border-width: 1px 0 0;
}
```

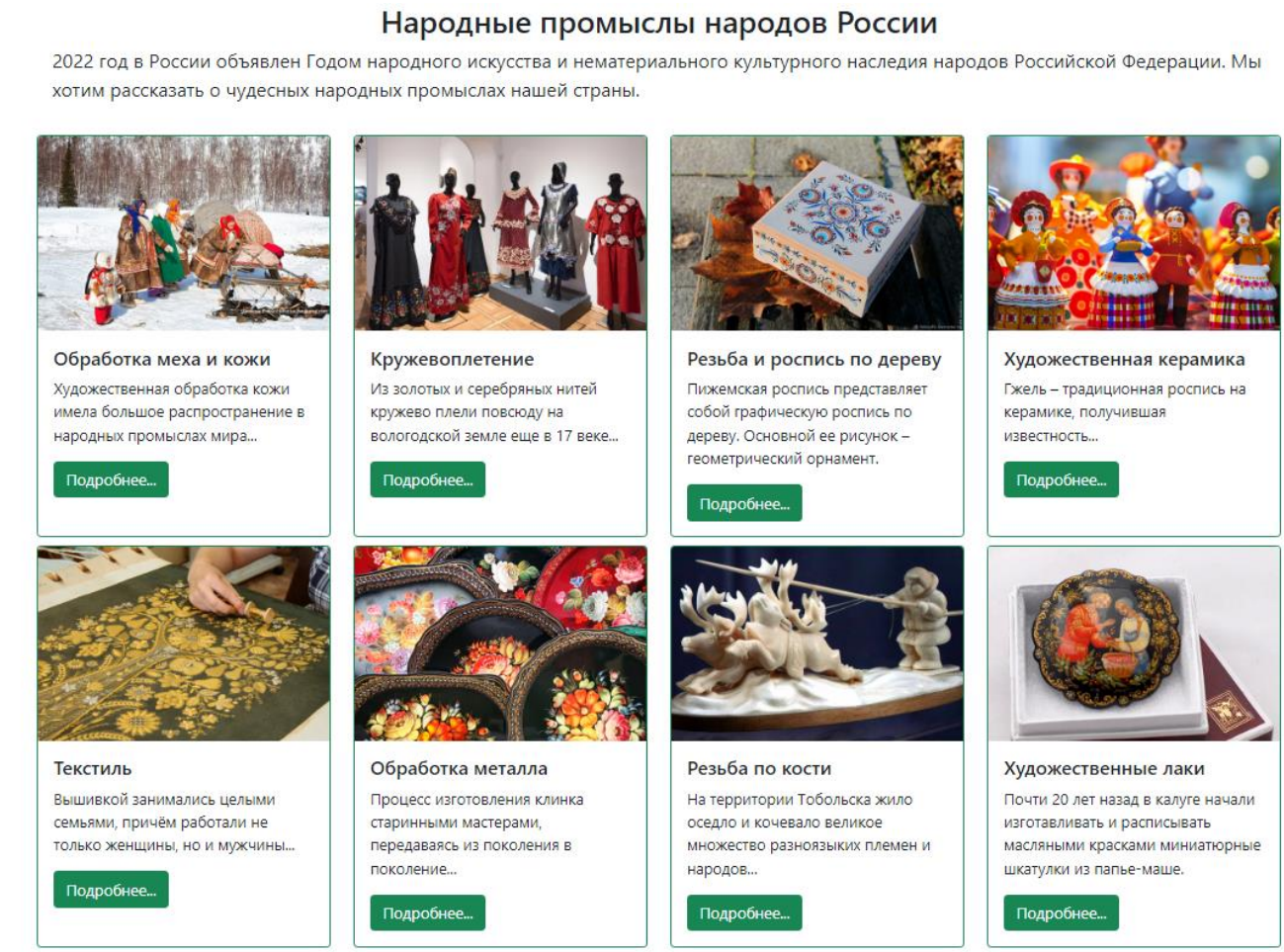
Второстепенные элементы макета, которые являются общими для всех компонентов страницы, также лучше настраивать через селекторы классов.

Если ваш стиль, определяемый через класс, относится элементу макета, рекомендуется добавить к его названию префикс **l-** или **layout-**. Это помогает легко определить назначение стилей и отделить их от модулей или состояний.

Самой сложной задачей становится проанализировать страницу и отделить части макета от модулей.

Например, карточки сами по себе представляют собой модули, а вот стили, отвечающие за их расположение относительно друг друга, относятся к настройкам макета.


©Pechora-PRO




Текущие элементы могут быть представлены в виде неупорядоченного списка, примерно такой структуры…

```
<div>
    <h2>Народные промыслы ...</h2>
    <p>...</p>
    <ul>
        <li>...</li>
        <li>...</li>
        ...
    </ul>
</div>
```

Без учета подхода SMACSS стили блока могли бы выглядеть следующим образом:

```
div#trade ul {
    margin: 0;
    padding: 0;
```


©Pechora-PRO

```
    list-style-type: none;
}

div#trade li {
    float: left;
    height: 100px;
    margin-left: 10px;
}
```

Те же настройки опишем в виде специальных классов и тогда, такой макет сетки может быть применен к любому контейнеру.

```
.l-grid {
    margin: 0;
    padding: 0;
    list-style-type: none;
}

.l-grid>li {
    display: inline-block;
    margin: 0 0 10px 10px;
}
```

Также мы снизили специфичность селекторов.

**Module / Модуль**

Модули — это компоненты пользовательского интерфейса, которые разделены и отличаются друг от друга. Обычно они находятся в компонентах макета, но иногда и внутри компонентов других модулей. На них не влияют другие модули или макеты пользовательского интерфейса. Примерами модулей могу быть, аккордеон, модальное окно или диалоговое окно, или карусель.

**Каждый модуль должен быть спроектирован так, чтобы существовать как отдельный компонент.** При этом страница станет более гибкой. Если все сделано правильно, модули можно легко перемещать в разные части макета, не нарушая при этом их оформления.


©Pechora-PRO

Добавляя классы к элементам, мы должны расширять семантику того, что означают эти элементы, и устранять любую двусмысленность, когда дело доходит до их стилизации.

**Важно!** Чем более семантически универсален HTML-элемент (например, span или div), тем больше вероятность того, что он создаст конфликт в будущем.

Например, у нас есть блок с текстовым содержимым …

```
<div class="fld">
    <span>Folder Name</span>
</div>
```

… и стили для их описания.

```
/* The Folder Module */
.fld>span {
    padding-left: 20px;
    background: url(icon.png);
}
```

Проблема в том, что по мере усложнения проекта увеличивается вероятность того, что вам понадобится расширить функциональность компонента, а значит, вы будете ограничены в использовании такого общего элемента в своем правиле.

Для такой конструкции стиль уже не подойдет.

```
<div class="fld">
    <span>Folder Name</span>
    <span>(32 items)</span>
</div>
```

А значит, мы должны разделить элементы span, не забывая учитывать их расположение.

```
<div class="fld">
    <span class="fld-name">Folder Name</span>
```


©Pechora-PRO

```
    <span class="fld-items">(32 items)</span>
</div>
```

### State / Состояние

Наборы CSS правил использующиеся для указания стилей для различных состояний компонента. По сути, это переопределение стиля пользовательского интерфейса по умолчанию.

Например, всплывающее сообщение может нести информацию об успешном или не успешном действии, а мы можем отразить это через цвет: красный или зелёный. **Состояния аналогичны модификаторам в методологии БЭМ.**

Создатель методологии рекомендует добавлять к таким стилям префикс is-{состояние}: is-active, is-error, is-focused и т.д.

В случае, когда правило состояния создано для конкретного модуля, имя класса состояния должно включать в себя имя модуля.

Правило состояния также должно быть размещено в коде вместе с правилами модуля, а не с остальными глобальными правилами состояния.

### Theme / Тема

Тема определяет цвета и изображения, которые задают вашему приложению или сайту определенные параметры оформления. Разделение темы на собственный набор стилей позволяет легко переопределять эти стили для альтернативных тем.


©Pechora-PRO

При этом темы могут влиять на любой из основных типов стилей. Она может переопределить базовые стили, такие как цвета ссылок по умолчанию. Она может изменять элементы модуля, такие как цвет границы. Она может повлиять на макет с различными вариантами расположения. Это также может изменить внешний вид элементов в определенном состоянии.

Можно добавлять именам классов, которые переопределяются темами отдельные префиксы и выносить их в отдельный файл.

**Важно! Важно!** В лекции представили краткую справку о методологии. Подробная информация на официальном портале https://smacss.com/


©Pechora-PRO